

STAT:5400 Section 3.1 — Random Number Generators

Boxiang Wang

2026-09-15

1 Principles of Simulation Studies

1.1 Reading

Art B. Owen. *Monte Carlo theory, methods and examples*.

<https://artowen.su.domains/mc/Ch-nonunifrng.pdf>

1.2 Computer simulation

- Computer simulations are experiments performed on the computer using **computer-generated random numbers**.
- Simulation is used to
 - study the behavior of **complex systems** such as biological systems, ecosystems, engineering systems, and computer networks, among many others;
 - compute values of otherwise **intractable quantities** such as integrals;
 - maximize or minimize the value of **complicated functions**;
 - study the behavior of statistical procedures;
 - implement novel methods of statistical inference.

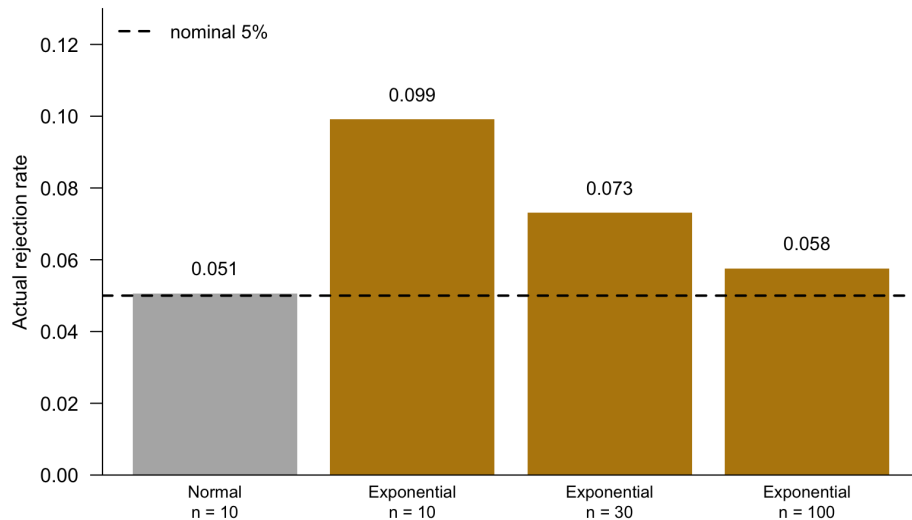
Source: https://homepage.divms.uiowa.edu/~luke/classes/STAT7400/modules/mod_sim.pdf

1.3 Simulations in statistics

- **Goal:** Simulation studies are commonly done to evaluate the performance of a frequentist statistical procedure, or to compare the performance of two or more different procedures for the same problem.
- Simulation studies enable us to see what happens “when many many samples of the same size are drawn from the same population.”
- **Why we need them:** properties of statistical methods must be established before the methods can safely be used in practice, but exact analytical derivations are rarely possible.
 - **Large sample approximations** are often available. Whether they are relevant at the (finite) sample sizes met in practice still has to be checked.
 - Analytical results may require **assumptions** such as normality. What happens when these assumptions are violated? Analytical results, even large sample ones, may not be possible.
- **Properties of estimators** that are often evaluated by simulations:
 - bias; mean squared error; coverage of confidence intervals.
- **Properties of hypothesis tests** also can be evaluated by simulation studies:
 - size; power.
- Simulation studies are experiments, and the things you know about experimental design and sample size calculation apply.

1.4 What happens when an assumption fails

The one-sample t -test is derived under normality. Below is how often a test set at the 5% level actually rejects a true null hypothesis when the data are exponential instead.



- With normal data the test does what it promises.
- With exponential data at $n = 10$ it rejects twice as often as it should.
- The error shrinks as n grows, but it is still there at $n = 100$.
- These numbers came from a simulation. You will learn how to produce them in this section.

1.5 Components in simulations

Simulations need

- uniform random numbers;
- non-uniform random numbers;
- random vectors, stochastic processes, etc;
- techniques to design good simulations;
- methods to analyze simulation results.

2 Uniform Random Numbers

2.1 Draw from the uniform distribution

To generate $U \sim \text{Unif}(0, 1)$, we call

```
(u <- runif(1))
```

- Note that U is a **random variable** which follows a uniform distribution on $(0, 1)$.
- In the example above, u is a **realization** of the random variable U .
- The realized value is obtained after the experiment was conducted.

2.2 Set seed

We need to set the seed to make our number generation **reproducible**. This is done with the `set.seed` function.

```
set.seed(5400)  
(u <- runif(1))
```

Run the block twice. The value does not change, because `set.seed` puts the generator back to the same starting state.

i Going further: what a seed actually is

R's numbers are not random. They are a long, fixed sequence that looks random, and the seed says where in that sequence to start.

R's default generator is the Mersenne-Twister. Its state is 625 integers, kept in `.Random.seed`. `set.seed(5400)` fills that state from the number 5400, so every later draw is determined. The sequence repeats only after $2^{19937} - 1$ numbers, which is why it looks random for any practical purpose.

```
set.seed(5400)  
length(.Random.seed)  
head(.Random.seed)
```

This is why a paper that reports a simulation must report its seed. Without the seed, the table cannot be reproduced.

2.3 Generate $U \sim \text{Unif}(1, 5)$

Recall that $U_1 \sim \text{Unif}(0, 1)$ and $U_2 = 1 + 4U_1$ give $U_2 \sim \text{Unif}(1, 5)$.

💡 Predict — two ways to reach the same distribution. Will they give the same number?

```
set.seed(5400)
(u1 <- runif(1, 1, 5))
```

```
set.seed(5400)
(u2 <- runif(1) * 4 + 1)
```

```
help(runif)
```

i Why the two lines agree

They agree exactly. `runif(1, 1, 5)` draws one number on $(0, 1)$ and then applies $1 + 4u$ internally.

This will not always happen. If a function uses a different number of uniform draws, or a different algorithm, the results differ. See **rexp** later on this page.

2.4 A table of random number generators

TABLES

TABLE B Random digits

Line								
101	19223	95034	05756	28713	96409	12531	42544	82853
102	73676	47150	99400	01927	27754	42648	82425	36290
103	45467	71709	77558	00095	32863	29485	82226	90056
104	52711	38889	93074	60227	40011	85848	48767	52573
105	95592	94007	69971	91481	60779	53791	17297	59335
106	68417	35013	15529	72765	85089	57067	50211	47487
107	82739	57890	20807	47511	81676	55300	94383	14893
108	60940	72024	17868	24943	61790	90656	87964	18883
109	36009	19365	15412	39638	85453	46816	83485	41979
110	38448	48789	18338	24697	39364	42006	76688	08708
111	81486	69487	60513	09297	00412	71238	27649	39950
112	59636	88804	04634	71197	19352	73089	84898	45785
113	62568	70206	40325	03699	71080	22553	11486	11776
114	45149	32992	75730	66280	03819	56202	02938	70915
115	61041	77684	94322	24709	73698	14526	31893	32592
116	14459	26056	31424	80371	65103	62253	50490	61181
117	38167	98532	62183	70632	23417	26185	41448	75532
118	73190	32533	04470	29669	84407	90785	65956	86382
119	95857	07118	87664	92099	58806	66979	98624	84826
120	35476	55972	39421	65850	04266	35435	43742	11937
121	71487	09984	29077	14863	61683	47052	62224	51025
122	13873	81598	95052	90908	73592	75186	87136	95761
123	54580	81507	27102	56027	55892	33063	41842	81868
124	71035	09001	43367	49497	72719	96758	27611	91596
125	96746	12149	37823	71868	18442	35119	62103	39244
126	96927	19931	36089	74192	77567	88741	48409	41903
127	43909	99477	25330	64359	40085	16925	85117	36071
128	15689	14227	06565	14374	13352	49367	81982	87209
129	36759	58984	68288	22913	18638	54303	00795	08727
130	69051	64817	87174	09517	84534	06489	87201	97245
131	05007	16632	81194	14873	04197	85576	45195	96565
132	68732	55259	84292	08796	43165	93739	31685	97150
133	45740	41807	65561	33302	07051	93623	18132	09547
134	27816	78416	18329	21337	35213	37741	04312	68508
135	66925	55658	39100	78458	11206	19876	87151	31260
136	08421	44753	77377	28744	75592	08563	79140	92454
137	53645	66812	61421	47836	12609	15373	98481	14592
138	66831	68908	40772	21558	47781	33586	79177	06928
139	55588	99404	70708	41098	43563	56934	48394	51719
140	12975	13258	13048	45144	72321	81940	00360	02428
141	96767	35964	23822	96012	94591	65194	50842	53372
142	72829	50232	97892	63408	77919	44575	24870	04178
143	88565	42628	17797	49376	61762	16953	88604	12724
144	62964	88145	83083	69453	46109	59505	69680	00900
145	19687	12633	57857	95806	09931	02150	43163	58636
146	37609	59057	66967	83401	60705	02384	90597	93600
147	54973	86278	88737	74351	47500	84552	19909	67181
148	00694	05977	19664	65441	20903	62371	22725	53340
149	71546	05233	53946	68743	72460	27601	45403	88692
150	07511	88915	41267	16853	84569	79367	32337	03316

Source: *The Practice of Statistics for Business and Economics*, 3rd ed.

Before computers, you looked numbers up in a printed table like this one. Starting at the same row gave the same numbers again, which is what `set.seed` does today.

2.5 Generating many uniforms

How about generating $U_1, \dots, U_{100} \stackrel{iid}{\sim} \text{Unif}(1, 5)$?

```
set.seed(5400)
(ulist <- runif(5, 1, 5))
```

The following code yields exactly the same results, but usually much slower.

```
set.seed(5400)
ulist <- rep(NA, 5)
for (i in seq(5)) {
  ulist[i] <- runif(1, 1, 5)
}
ulist
```

2.6 R and Python do not share a stream

R and Python do not generate the same random numbers even if the same seed is used.

```
import numpy as np

np.random.seed(5400)
print(4 * np.random.uniform(0, 1, 1) + 1)
```

```
np.random.seed(5400)
print(np.random.uniform(1, 5, 5))
```

R and NumPy use different generators, so the same seed gives different numbers.

Calling R from Python

If you need R's exact numbers inside Python, call R.

```
import rpy2.robjects as robjects
ulist = robjects.r("""
set.seed(5400)
ulist <- runif(5, 1, 5)
""")
print(np.array(ulist))
# [4.86860625 4.40436475 4.62585277 2.63676882 4.37947794]
```

Install the module with `pip install rpy2` (or `conda install rpy2`). It needs a real R on the machine, so it does not run in the browser.

2.7 Mean and variance of uniform distribution

Given $U \sim \text{Unif}(0, a)$, it is known that $E(U) = a/2$ and $\text{Var}(U) = a^2/12$. What are the **sample estimates** of those two **population parameters**?

2.7.1 R

```
set.seed(5400)
# generate U1, U2, ..., U100
ulist <- runif(100, 0, 1)
```

```
# population mean is 1/2
mean(ulist)
```

```
# population variance is 1/12 = 0.0833
var(ulist)
```

2.7.2 Python

```
import numpy as np

np.random.seed(5400)
ulist = np.random.uniform(0, 1, 100)
print(np.mean(ulist))
print(np.var(ulist))
```

```
import scipy.stats

np.random.seed(5400)
ulist = scipy.stats.uniform.rvs(0, 1, 100)
print(ulist.mean())
print(ulist.var())
```

 Edit — what happens if the sample size increases? Change 100 to 10000 and run again.

Going further: two different variances

R's `var` divides by $n - 1$; NumPy's `np.var` divides by n .

R returns the unbiased sample variance S^2 . NumPy returns the plug-in estimate, which is biased low by a factor $(n - 1)/n$. With $n = 100$ the two differ by 1%.

To make NumPy match R, pass `ddof=1`:

```
np.var(ulist, ddof=1)
```

We meet this same n versus $n - 1$ choice again in Section 4.1, where the plug-in estimate is the natural one.

2.8 Estimate the mean and variance

Let $X_1, \dots, X_n$ be a random sample from a distribution with mean μ and variance σ^2 . Let

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i, \quad S^2 = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2.$$

The **statistics** $\bar{X}$ and S^2 are **unbiased** and **consistent** estimators of the **parameters** μ and σ^2 , respectively:

$$E\bar{X} = \mu, \quad ES^2 = \sigma^2, \quad \bar{X} \xrightarrow{P} \mu, \quad S^2 \xrightarrow{P} \sigma^2.$$

2.9 Did we really generate from F_X ?

- We have generated U from the uniform distribution.
- Let us think about it broadly: say we claim we have generated X from a distribution with CDF F_X .
- **Question:** How can we verify that X is truly from the distribution with CDF F_X ?

To deal with this question, let us first present an important fact: $Y \sim \text{Unif}(0, 1)$ if $Y = F(X)$.

For simplicity, assume F is continuous and strictly increasing (this can be dropped).

$$P(Y \leq y) = P(F(X) \leq y) = P(X \leq F^{-1}(y)) = F(F^{-1}(y)) = y.$$

2.10 The probability integral transform, seen

2.10.1 R

```
par(mfrow = c(1, 2))
X <- runif(1000, 2, 5)
hist(punif(X, 2, 5), main = "Uniform CDF")
X <- rnorm(1000)
hist(pnorm(X), main = "Normal CDF")
```

2.10.2 Python

```
import matplotlib.pyplot as plt
import scipy.stats

plt.subplot(1, 2, 1)
X = np.random.uniform(2, 5, 1000)
plt.hist(scipy.stats.uniform.cdf(X, loc=2, scale=3))
plt.title("Uniform CDF")
plt.subplot(1, 2, 2)
X = scipy.stats.norm.rvs(0, 1, 1000)
plt.hist(scipy.stats.norm.cdf(X, loc=0, scale=1))
plt.title("Normal CDF")
plt.show()
```

Both histograms are flat: $F(X)$ is uniform for both distributions.

2.11 Uniform random number generators in Python

Two generators have different arguments: `low-high` versus `loc-scale`.

```
np.random.seed(5400)
print(np.random.uniform(low=2, high=5, size=3))
```

```
np.random.seed(5400)
print(scipy.stats.uniform.rvs(loc=2, scale=3, size=3))
```

`scale` is the width, not the upper end. To get $\text{Unif}(2, 5)$ you write `scale=3`, because $5 - 2 = 3$. Passing `scale=5` silently gives you $\text{Unif}(2, 7)$.

2.12 Q-Q plot (quantile-quantile plot)

- Check whether the sample conforms to a specific distribution.
- CDF: $P(X \leq a) = F_X(a)$.
- A number a is called a p th quantile of F_X if $F_X(a) = p$.
- Suppose a sample $X_1, \dots, X_n$ follows F_X and sort the data into ascending order $X_{(1)}, \dots, X_{(n)}$.
- If the sample is large, we would expect

$$F_X(X_{(i)}) \approx \frac{i - 0.5}{n}.$$

To be consistent with R, we use something equivalent:

$$X_{(i)} \approx F_X^{-1} \left(\frac{i - 0.5}{n} \right).$$

- Plot of $X_{(i)}$ against $F_X^{-1} \left(\frac{i-0.5}{n} \right)$ should be an approximately straight line with slope 1 and intercept 0.
- We make this plot. If it seems linear, that supports the model that the data follow a distribution with CDF F_X .

```
set.seed(5400)
ulist <- runif(1000, 0, 1)
plot(qunif(ppoints(1000)), sort(ulist),
     xlab = "Theoretical Quantiles",
     ylab = "Sample Quantiles", main = "Q-Q plot")
abline(0, 1)
```

3 Bernoulli and Binomial Distributions

3.1 Drawing from Bernoulli distribution

- How to generate $Y \sim \text{Bern}(p)$, whose PMF is $P(Y = 1) = p$ and $P(Y = 0) = 1 - p$, with $0 < p < 1$?
- Generate $U \sim \text{Unif}(0, 1)$. Let $Y = I(U < p)$, thus $Y \sim \text{Bern}(p)$.
- Algorithm:
 1. Generate $U \sim \text{Unif}(0, 1)$.
 2. Compare U and p . Let $Y = 1$ if $U < p$ and $Y = 0$ otherwise.

```
# Generate Bern(0.4)
Y <- as.numeric(runif(1) < 0.4)
Y
```

```
set.seed(5400)
# Generate Y1, ..., Y100 ~ Bern(0.4)
Ylist <- as.numeric(runif(100) < 0.4)
```

```
# population mean is 0.4
mean(Ylist)
```

```
# population variance is 0.4 * 0.6 = 0.24
var(Ylist)
```

3.2 Drawing from Binomial distribution

- How to generate $X \sim \text{Bin}(n, p)$?
- Generate $Y_1, \dots, Y_n \sim \text{Bern}(p)$. Let $X = \sum_{i=1}^n Y_i$.

```
set.seed(5400)
# Generate X ~ Bin(100, 0.4)
X <- sum(runif(100) < 0.4)
X
```

In R,

```
rbinom(1, 100, 0.4)
# rbinom(n, size, prob).
# n is the number of observations.
# size is the number of trials.
```

In Python,

```
np.random.seed(5400)
print(np.random.binomial(n=100, p=0.4, size=1))
print(scipy.stats.binom.rvs(n=100, p=0.4, size=1))
```

4 Inverse CDF Method

4.1 Inverse CDF method

- Suppose F_X is a cumulative distribution function. **Goal: generate $X \sim F_X$.**
- Suppose that F_X is continuous and increasing. Let F^{-1} be its inverse function. If $U \sim \text{Unif}(0, 1)$, then $X = F_X^{-1}(U)$ has CDF F .
- Proof:

$$\begin{aligned}
P(F_X^{-1}(U) \leq t) &= P(F_X(F_X^{-1}(U)) \leq F_X(t)) \\
&= P(U \leq F_X(t)) \\
&= F_X(t),
\end{aligned}$$

which says the random variable $X = F^{-1}(U)$ has CDF F .

- In fact, **the assumption that F_X is continuous and increasing can be dropped**. The inverse CDF method is always valid based on the fact that F_X is right continuous.

4.2 Drawing from exponential distribution

- The CDF for the **unit** exponential distribution $\exp(1)$ is $F(t) = 1 - \exp(-t)$, $t \geq 0$.
- The inverse function is $F^{-1}(u) = -\log(1 - u)$.
- Given $U \sim \text{Unif}(0, 1)$, $X = -\log(1 - U)$ has an exponential distribution.
- Since $1 - U \sim \text{Unif}(0, 1)$, $-\log(U)$ also has a unit exponential distribution.
- How to generate $\exp(\lambda)$ where λ is the mean?

4.2.1 R

```
set.seed(5400)
# Generate X1, ..., X1000 ~ exp(2)
Ulist <- runif(1000, 0, 1)
Xlist <- -2 * log(Ulist)
Xlist[1:3]
```

```
mean(Xlist)
var(Xlist)
```

4.2.2 Python

```
np.random.seed(5400)
Ulist = np.random.uniform(0, 1, 1000)
Xlist = [-2 * np.log(u) for u in Ulist]
print(Xlist[0:3])
```

```
print(np.mean(Xlist))
print(np.var(Xlist))
```

The mean is near 2 and the variance near 4, which is what $\exp$ with mean 2 should give.

4.3 Histogram against the density

👉 By hand — fill in the density call so the curve matches the histogram. Done when the curve sits on top of the bars. No AI for this block.

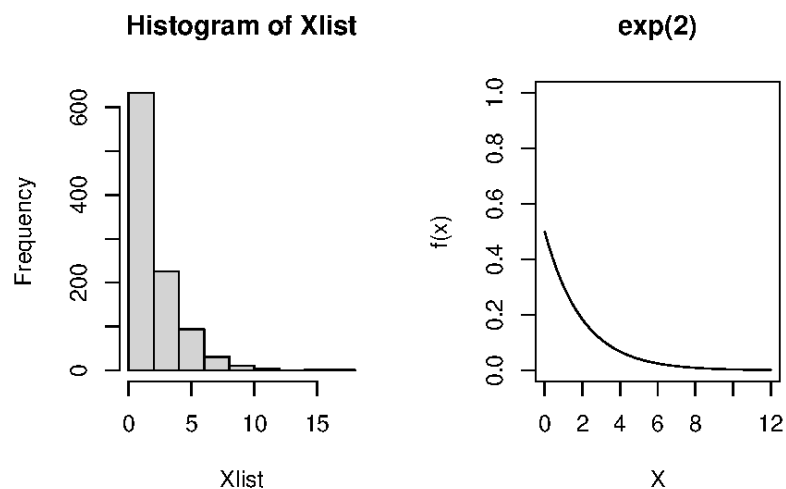
```
par(mfrow = c(1, 2))
hist(Xlist)
xs <- seq(0.001, 12, len = 200)
plot(xs, dexp(xs, rate = 1),          # <- change this line
     type = "l", ylim = c(0, 1),
     xlab = "X", main = "exp(2)", ylab = "f(x)")
```

i Solution

The rate is the reciprocal of the mean. Xlist has mean 2, so the rate is $1/2$:

```
plot(xs, dexp(xs, rate = 1/2), type = "l", ylim = c(0, 1),
     xlab = "X", main = "exp(2)", ylab = "f(x)")
```

R's `dexp`, `rexp`, `pexp` and `qexp` are all parameterized by **rate**, not mean. Python's `np.random.exponential` is parameterized by **scale**, which is the mean. This is a common mistake when translating between the two languages.



4.4 Q-Q plot for the exponential

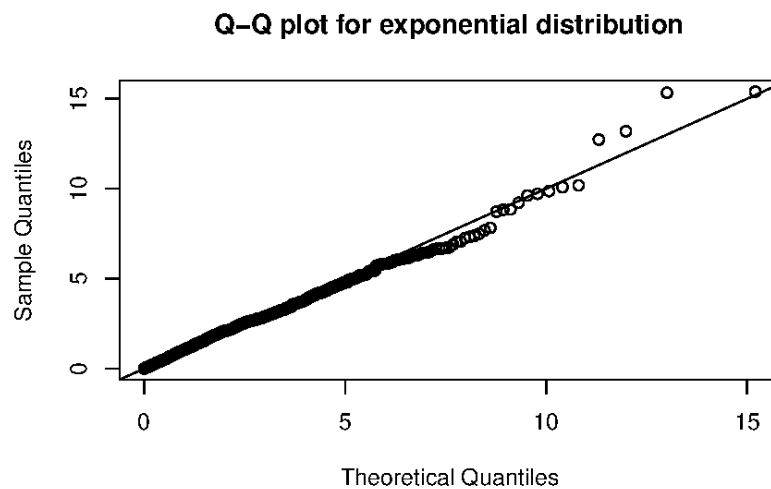
👉 By hand — fill in the two arguments of `plot` so the points fall on the line. Done when the cloud follows the 45-degree line. No AI for this block.

```
probs <- ppoints(1000)
plot(probs, probs,                      # <- change this line
     xlab = "Theoretical Quantiles", ylab = "Sample Quantiles",
     main = "Q-Q plot for exponential distribution")
abline(0, 1)
```

i Solution

```
plot(qexp(probs, rate = 0.5), sort(Xlist),
     xlab = "Theoretical Quantiles", ylab = "Sample Quantiles",
     main = "Q-Q plot for exponential distribution")
abline(0, 1)
```

The theoretical quantiles go on the x-axis and the **sorted** sample on the y-axis. Forgetting **sort** is the usual mistake, and it produces a shapeless cloud rather than a line.



4.5 The rexp function in R

```
set.seed(5400)
Xlist <- rexp(1000, rate = 1/2)
Xlist[1:3]
```

```
mean(Xlist)
var(Xlist)
```

R does not implement `rexp` based on the inverse CDF method. The algorithm was developed in Marsaglia (1961), *Generating exponential random variables*, **Annals of Mathematical Statistics**, 899–902.

Source code: <https://github.com/wch/r-source/blob/trunk/src/nmath/sexp.c>

💡 Predict — the same seed, the same distribution, two algorithms. Do the first three numbers match the inverse-CDF ones above?

```
set.seed(5400)
head(-2 * log(runif(3)))
set.seed(5400)
head(rexp(3, rate = 1/2))
```

i Why they differ

They do not match. Both are correct draws from the same distribution, but they consume the uniform stream differently.

The inverse-CDF version uses exactly one uniform per exponential. Marsaglia's algorithm uses a variable number, sometimes one and sometimes several. So after the first value the two streams are at different places in the same sequence.

The seed fixes the sequence of uniforms. It does not fix the result if the algorithm, or the order of the draws, changes.

4.6 Drawing from the standard Cauchy distribution

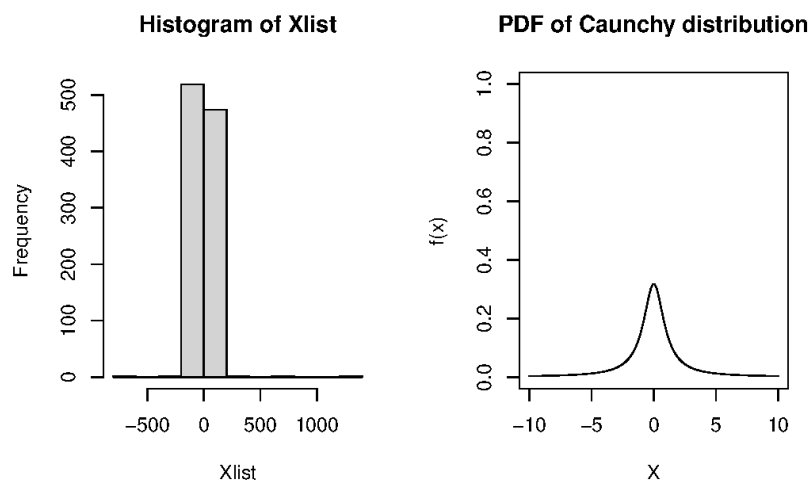
- The **standard Cauchy distribution** is equivalent to the t -distribution with one degree of freedom. Its CDF is

$$F(x) = \frac{1}{2} + \frac{1}{\pi} \arctan(x).$$

- The inverse CDF is $F^{-1}(u) = \tan(\pi(u - 1/2))$. Thus $X = \tan(\pi(U - 0.5))$ has a standard Cauchy distribution.
- Try the `rcauchy` function in R.

```
set.seed(5400)
# Generate X1, ..., X1000 ~ Cauchy
Ulist <- runif(1000, 0, 1)
Xlist <- tan(pi * (Ulist - 0.5))
```

```
par(mfrow = c(1, 2))
hist(Xlist)
xs <- seq(-10, 10, len = 200)
plot(xs, dcauchy(xs), type = "l", ylim = c(0, 1),
     xlab = "X", main = "PDF of Cauchy distribution", ylab = "f(x)")
```



👉 By hand — fill in the Q-Q plot for the Cauchy sample. Done when the middle of the cloud is straight. No AI for this block.

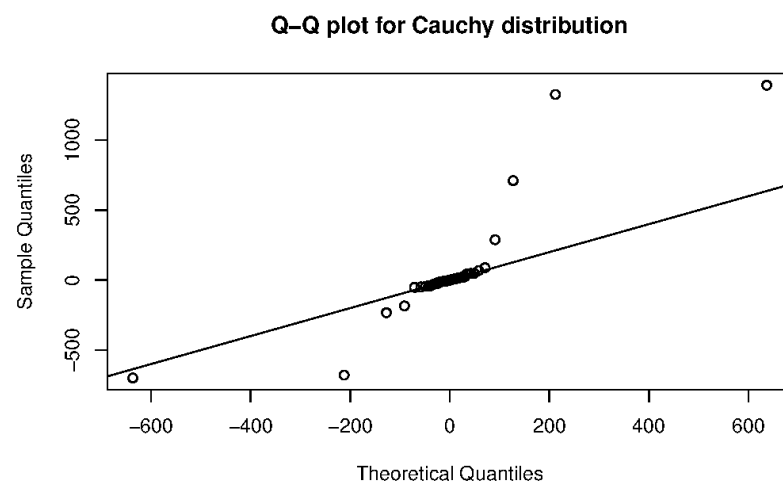
```
probs <- ppoints(1000)
plot(probs, probs,
     xlab = "Theoretical Quantiles",
     ylab = "Sample Quantiles",
     main = "Q-Q plot for Cauchy distribution")
abline(0, 1)
```

i Solution

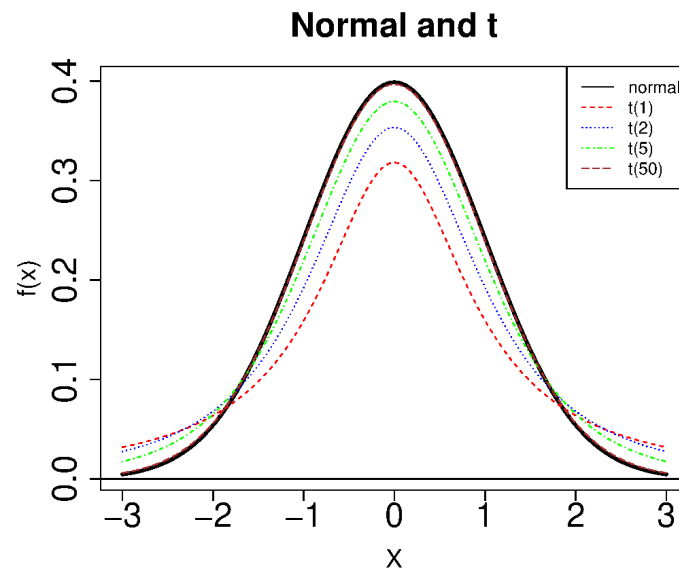
```
plot(tan(pi * (probs - 0.5)), sort(Xlist),  
     xlab = "Theoretical Quantiles", ylab = "Sample Quantiles",  
     main = "Q-Q plot for Cauchy distribution")  
abline(0, 1)
```

You may also write `qcauchy(probs)` in place of `tan(pi * (probs - 0.5))`. They are the same function.

The extreme points are very far from the line. The Cauchy has heavy tails, so a few points are very far out.



4.7 Cauchy and t



- The Cauchy distribution is a special case of the t -distribution with 1 degree of freedom.
- The Cauchy distribution is widely used to exemplify **heavy-tailed data** in simulations.

```
set.seed(5400)
mean(rcauchy(10000))
mean(rcauchy(10000))
mean(rcauchy(10000))
```

The three means are far apart, although each uses 10,000 draws. The sample mean does not settle down, because the Cauchy has no mean.

4.8 Sampling the next word

A language model draws its next word from a discrete distribution. Take the sentence

The p-value is 0.03, so at the 5% level we ____ the null hypothesis.

Suppose the model gives the probabilities below. We draw one word with the inverse CDF method, now for a discrete distribution: find where one uniform falls in the cumulative sums.

4.8.1 R

```
words <- c("reject", "accept", "retain")
p <- c(0.665, 0.245, 0.090)

set.seed(5400)
u <- runif(1)
words[findInterval(u, cumsum(p)) + 1] # one uniform, one word
```

```
# 10,000 draws: the frequencies match p
u <- runif(10000)
table(words[findInterval(u, cumsum(p)) + 1]) / 10000
```

4.8.2 Python

```
import numpy as np
words = ["reject", "accept", "retain"]
p = np.array([0.665, 0.245, 0.090])

rng = np.random.default_rng(5400)
u = rng.random()
print(words[np.searchsorted(np.cumsum(p), u, side="right")]) # one uniform, one word

# 10,000 draws: the frequencies match p
u = rng.random(10000)
idx = np.searchsorted(np.cumsum(p), u, side="right")
print(np.bincount(idx) / 10000)
```

- The model does not always return the most likely word. This is why the same question asked twice can give two different answers.
- Temperature and top-p change the vector $\mathbf{p}$ before the draw. See [Section 2.4, Sampling controls](#).

5 Accept-Reject Sampling

5.1 Accept/reject sampling

- How to generate (V_1, V_2) that is uniformly distributed in a unit square, i.e., $-1 \leq V_1, V_2 \leq 1$?

- Independently generate $V_1, V_2 \sim \text{Unif}(-1, 1)$, with `runif(2, -1, 1)`.
- How to generate (V_1, V_2) that is uniformly distributed in a unit disk, i.e., $V_1^2 + V_2^2 \leq 1$?
- **Algorithm:**
 - repeat
 - * independently generate $V_1, V_2 \sim \text{Unif}(-1, 1)$
 - until $V_1^2 + V_2^2 \leq 1$
 - return (V_1, V_2)

5.2 Accept/reject sampling (cont'd)

- This independently generates pairs (V_1, V_2) uniformly on the square $(-1, 1) \times (-1, 1)$ until the result is inside the unit disk.
- The resulting pair is uniformly distributed on the unit disk.
- Probability of acceptance is

$$\frac{\text{area of disk}}{\text{area of square}} = \frac{\pi}{4}.$$

- This approach is the **accept-reject sampling** method. More details and applications will be discussed in STAT 7400. <https://homepage.divms.uiowa.edu/~luke/classes/STAT7400/notes.pdf>
- What is the distribution of V_1/V_2 ?

5.2.1 R

```
set.seed(5400)
n <- 2000      # total samples
# Each row is a pair of V1 and V2. n rows in total.
dat <- matrix(runif(n * 2, -1, 1), n, 2)
# Accept if V_1^2 + V_2^2 <= 1.
accept <- (dat[, 1]^2 + dat[, 2]^2) <= 1
```

```
mean(accept)   # which is about pi/4
pi / 4
```

```
# Accepted pairs of V1 and V2
V <- dat[accept, ]
# Rejected pairs of V1 and V2
V_out <- dat[!accept, ]
```

```
# draw a circle
theta <- seq(0, 2 * pi, len = 1000)
plot(cbind(sin(theta), cos(theta)),
     type = "l", xlab = "x", ylab = "y")
points(V, pch = 2, col = "blue")      # Accepted pairs
points(V_out, pch = 1, col = "red")   # Rejected pairs
```

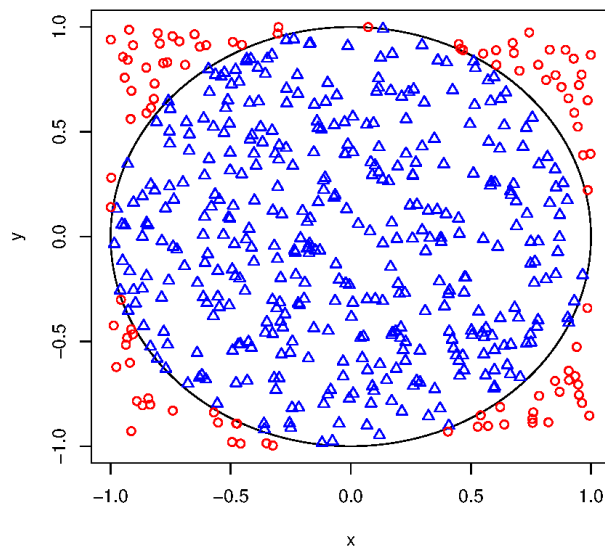
5.2.2 Python

```
np.random.seed(5400)
n = 500 # total samples
# Each row is a pair of V1 and V2. n rows in total.
dat = [np.random.uniform(-1, 1, 2) for i in range(n)]
# Accept if  $V_1^2 + V_2^2 \leq 1$ .
accept = [(dat[i][0]**2 + dat[i][1]**2) <= 1 for i in range(n)]
print(np.mean([int(x) for x in accept]), np.pi / 4)
```

```
# Accepted pairs of V1 and V2
V = [dat[i] for i in range(n) if accept[i]]
# Rejected pairs of V1 and V2
V_out = [dat[i] for i in range(n) if not accept[i]]
```

```
import matplotlib.pyplot as plt

v = np.arange(-np.pi, np.pi, 0.1)
plt.plot(np.sin(v), np.cos(v), linewidth=0.5)
plt.scatter([row[0] for row in V], [row[1] for row in V],
            color="blue", marker="^", s=8)
plt.scatter([row[0] for row in V_out], [row[1] for row in V_out],
            color="red", marker="o", s=8)
plt.axis('equal')
plt.show()
```



Since the acceptance rate estimates $\pi/4$, $4 * \text{mean}(\text{accept})$ estimates π . This is Monte Carlo integration. Its error decreases like $1/\sqrt{n}$.

i Going further: the general accept-reject method

The disk inside the square is one case of a general recipe. Sample from an easy density that sits above the hard one, then keep each draw with probability equal to the ratio of the two.

- You want to sample from f , the **target** density. You can already sample from g , the **envelope** density.
- Find a constant c with $f(x) \leq c g(x)$ for every x .
- **Algorithm:**
 - repeat
 - * draw $Y \sim g$ and $U \sim \text{Unif}(0, 1)$
 - until $U \leq f(Y)/(c g(Y))$
 - return Y
- The accepted Y has density f exactly, not approximately.
- Each trial is accepted with probability $1/c$, so the expected number of trials is c . So we want c as small as possible.
- The disk is this recipe with f uniform on the disk, g uniform on the square, and $c = 4/\pi = 1.27$.

A worked case: draw standard normals using a Cauchy envelope. The Cauchy has heavier tails than the normal, so a single constant does bound the ratio. Here $c = 1.52$, and about two thirds of the draws survive. The shaded gap between the two curves in the plot below is the part that is rejected. A better envelope makes it smaller.

5.2.3 R

```
c_bound <- optimize(function(x) dnorm(x) / dcauchy(x),
                    c(-3, 3), maximum = TRUE)$objective
c_bound
```

```
xs <- seq(-6, 6, length = 400)
plot(xs, c_bound * dcauchy(xs), type = "n",
     xlab = "x", ylab = "density", ylim = c(0, 0.52))
polygon(c(xs, rev(xs)),
       c(c_bound * dcauchy(xs), rev(dnorm(xs))),
       col = "gray90", border = NA) # rejected
lines(xs, c_bound * dcauchy(xs), lwd = 2, col = "#b8860b")
lines(xs, dnorm(xs), lwd = 2)
legend("topright", bty = "n", lwd = 2,
      col = c("#b8860b", "black"),
      legend = c("c g(x), Cauchy envelope", "f(x), standard normal"))
```

```
set.seed(5400)
n <- 1e4
Y <- rcauchy(n)
U <- runif(n)
keep <- U <= dnorm(Y) / (c_bound * dcauchy(Y))

mean(keep)      # close to 1 / c_bound
1 / c_bound
```

```
X <- Y[keep]
hist(X, breaks = 40, freq = FALSE, main = "", xlab = "x")
curve(dnorm(x), add = TRUE, lwd = 2)
```

5.2.4 Python

```
from scipy import stats, optimize
c_bound = -optimize.minimize_scalar(
    lambda x: -stats.norm.pdf(x) / stats.cauchy.pdf(x),
    bounds=(-3, 3), method="bounded").fun
print(c_bound)
```

```
xs = np.linspace(-6, 6, 400)
env = c_bound * stats.cauchy.pdf(xs)
tgt = stats.norm.pdf(xs)
plt.fill_between(xs, tgt, env, color="0.9")      # rejected
plt.plot(xs, env, lw=2, color="#b8860b", label="c g(x), Cauchy envelope")
plt.plot(xs, tgt, lw=2, color="black", label="f(x), standard normal")
plt.xlabel("x"); plt.ylabel("density"); plt.legend()
plt.show()
```

```
rng = np.random.default_rng(5400)
n = 10000
Y = rng.standard_cauchy(n)
U = rng.random(n)
keep = U <= stats.norm.pdf(Y) / (c_bound * stats.cauchy.pdf(Y))
print(keep.mean(), 1 / c_bound)
```

```
X = Y[keep]
plt.hist(X, bins=40, density=True)
xs = np.linspace(-4, 4, 200)
plt.plot(xs, stats.norm.pdf(xs), lw=2)
plt.show()
```

Squeezing, the discrete case, the ratio-of-uniforms method, and adaptive rejection sampling all build on this same picture. They are covered in STAT 7400.

Source: the treatment follows Luke Tierney, *Simulation*, STAT:7400 course module, pages 37 to 54. https://homepage.divms.uiowa.edu/~luke/classes/STAT7400/modules/mod_sim.pdf

6 Multivariate Transformations

6.1 Box-Muller method for the standard normal

If U_1 and U_2 are independent and uniform on $[0, 1]$, then

$$\begin{aligned}X_1 &= \sqrt{-2 \log U_1} \cos(2\pi U_2), \\X_2 &= \sqrt{-2 \log U_1} \sin(2\pi U_2),\end{aligned}$$

then X_1 and X_2 are independent standard normals.

- How about the univariate normal distribution $Z \sim N(0, 1)$?
- How about $N(\mu, \sigma^2)$?
- Try the `rnorm` function in R.

6.2 Polar method for the standard normal

- The polar method (by Marsaglia and Bray) is an alternative algorithm to generate the standard normal, and it avoids the slow computation caused by the trigonometric functions.
- Generate (V_1, V_2) which is uniform on the unit disk. Let $R^2 = V_1^2 + V_2^2$. Then

$$\begin{aligned}X_1 &= V_1 \sqrt{-(2 \log R^2)/R^2}, \\X_2 &= V_2 \sqrt{-(2 \log R^2)/R^2},\end{aligned}$$

are independent standard normals.

6.2.1 R

```
# an OK code
Rsq <- V[, 1]^2 + V[, 2]^2
m <- sqrt(-(2 * log(Rsq)) / (Rsq))
X <- m * V
```

```
# a better code
Rsq <- V[, 1]^2 + V[, 2]^2
Rsq <- Rsq + .Machine$double.xmin
m <- sqrt(-(2 * log(Rsq)) / (Rsq))
X <- m * V
```

6.2.2 Python

```
import sys

Rsq = [row[0]**2 + row[1]**2 for row in V]
Rsq = np.array(Rsq) + sys.float_info.min
m = [np.sqrt(-(2 * np.log(x)) / x) for x in Rsq]
x = [m[i] * V[i] for i in range(len(V))]
```

💡 Predict — why is the second version better? What does `.Machine$double.xmin` protect against?

i What the tiny number is for

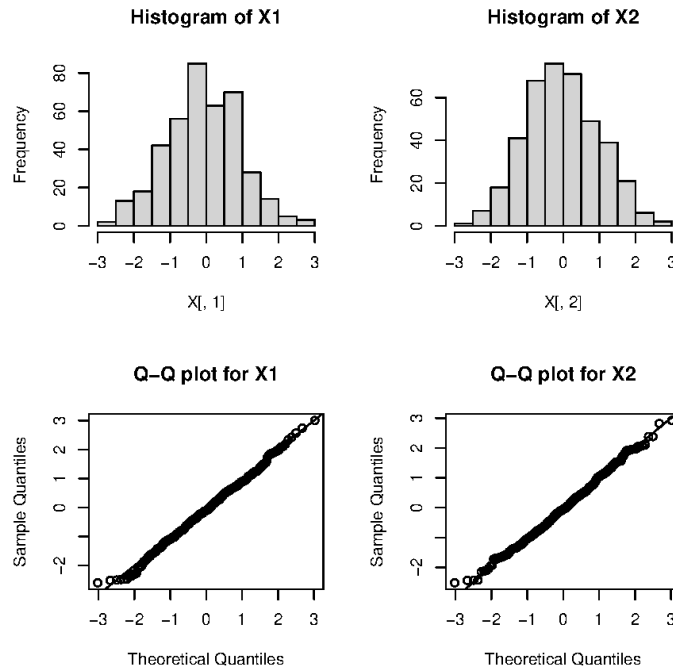
It stops `log(0)`. If an accepted pair is exactly $(0, 0)$, `Rsq` is 0, `log(0)` is $-\text{Inf}$, and the result is `NaN`.

Adding `.Machine$double.xmin`, the smallest positive double (about $2.2\text{e-}308$), moves it off zero by an amount too small to change any other value. In general, check where a formula divides by zero or takes the log of zero, and protect that step.

This almost never happens in one run, so tests rarely catch it. In a long simulation it can happen.

6.3 Checking the normals

```
par(mfrow = c(2, 2))
# histograms
hist(X[, 1], main = "Histogram of X1")
hist(X[, 2], main = "Histogram of X2")
# Q-Q plots
probs <- ppoints(NROW(X))
plot(qnorm(probs), sort(X[, 1]),
     xlab = "Theoretical Quantiles",
     ylab = "Sample Quantiles", main = "Q-Q plot for X1")
abline(0, 1)
plot(qnorm(probs), sort(X[, 2]),
     xlab = "Theoretical Quantiles",
     ylab = "Sample Quantiles", main = "Q-Q plot for X2")
abline(0, 1)
```



6.4 Q-Q plot for $N(\mu, \sigma^2)$

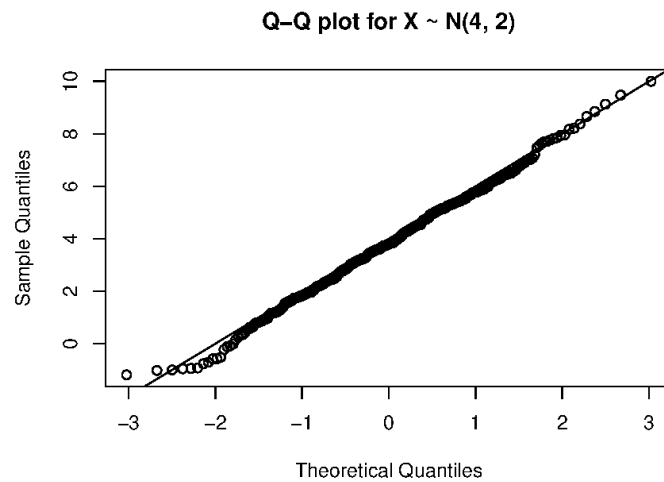
How to generate $X \sim N(\mu, \sigma^2)$?

- Generate $Z \sim N(0, 1)$.
- Compute $X = \mu + \sigma Z$.

How about the Q-Q plot?

- Plot of $X_{(i)}$ against $\text{qnorm}\left(\frac{i-0.5}{n}\right)$ should be an approximately straight line with slope σ and intercept μ .

```
newX <- X[, 1] * 2 + 4
dat_quant <- sort(newX)
thr_quant <- qnorm(ppoints(length(newX)))
plot(thr_quant, dat_quant,
     xlab = "Theoretical Quantiles",
     ylab = "Sample Quantiles",
     main = "Q-Q plot for X ~ N(4, 2)")
abline(a = 4, b = 2) # a = intercept; b = slope
```



6.5 qqnorm and qqline

- In practice, we want to check the normality.
- In such a case, how can we get the slope and intercept?
- Check out the R functions `qqnorm` and `qqline`.

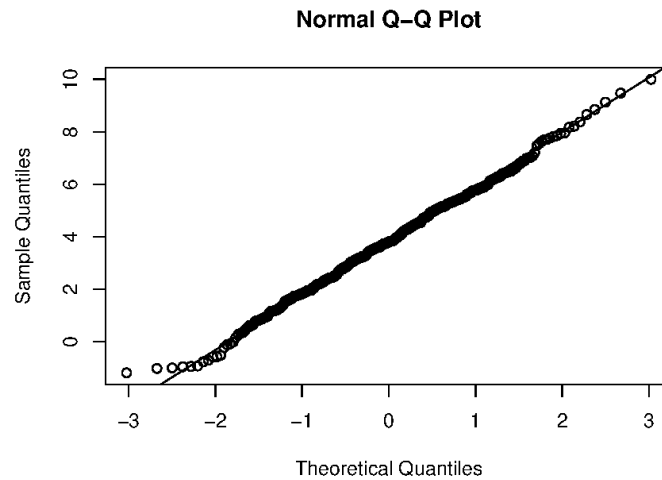
6.5.1 R

```
qqnorm(newX)
qqline(newX)
```

6.5.2 Python

```
import scipy.stats
import matplotlib.pyplot as plt

newx = np.array(x)[: , 1] * 2 + 4
fig, ax = plt.subplots(1, 1)
scipy.stats.probplot(newx, plot=plt)
plt.show()
```



`qqline` does not use the mean and standard deviation. It draws the line through the first and third sample quartiles, which is more resistant to a few outliers than a fitted least-squares line would be.

6.6 What R actually uses

- Neither Box-Muller nor the polar method is the most efficient. They are useful when a pair of normal variables is generated.
- The default method in R to generate normal distributions is through the inverse CDF method (AS 241 algorithm): Wichura, M. J. (1988) *Algorithm AS 241: The percentage points of the normal distribution*, **Applied Statistics**, 37, 477–484.

```
set.seed(5400)
qnorm(runif(1))
```

```
set.seed(5400)
rnorm(1)
```

The two agree to about 7 digits. R uses inversion, but it combines two uniforms for each normal to get more precision, so the numbers are not exactly equal.

6.7 Standard Cauchy, again

- If X_1 and X_2 are independent standard normals, then $Y = X_1/X_2$ follows a standard Cauchy distribution.
- Let us continue the previous example. Since

$$X_1 = V_1 \sqrt{-(2 \log R^2)/R^2}, \quad X_2 = V_2 \sqrt{-(2 \log R^2)/R^2},$$

then

$$Y = \frac{X_1}{X_2} = \frac{V_1}{V_2}$$

follows the standard Cauchy distribution. The messy square root cancels, so the ratio of the two accepted uniforms is already Cauchy.

```
Y <- V[, 1] / V[, 2]
summary(Y)
```

6.8 χ_v^2 distribution

Suppose $Z_1, \dots, Z_v$ are independent standard normals. Then

$$Y = \sum_{i=1}^v Z_i^2$$

has a χ_v^2 distribution.

```
set.seed(5400)
Zlist <- rnorm(10)
Y <- sum(Zlist^2)
Y
```

- Note this is not an efficient way to generate χ_v^2 .
- A better way is to generate a gamma distribution, since χ_v^2 is a $\text{Gamma}(v/2, 2)$ distribution.
- A gamma distribution can be generated through accept-reject sampling.

6.9 Student's t distribution

Suppose

- Z has a standard normal distribution,
- Y has a χ_v^2 distribution,
- Z and Y are independent.

Then

$$X = \frac{Z}{\sqrt{Y/v}}$$

has a t distribution with v degrees of freedom.

```
curve(dnorm(x, 0, 1), xlim = c(-3, 3),
      main = "Normal and t",
      xlab = "X", ylab = "f(x)", lwd = 3, lty = 1)
curve(dt(x, 1), xlim = c(-3, 3), add = TRUE, lty = 2, col = "red")
curve(dt(x, 2), xlim = c(-3, 3), add = TRUE, lty = 3, col = "blue")
curve(dt(x, 5), xlim = c(-3, 3), add = TRUE, lty = 4, col = "green")
abline(h = 0)
legend("topright",
      c("normal", "t(1)", "t(2)", "t(5)"),
      lty = 1:4,
      col = c("black", "red", "blue", "green"))
```

6.10 F distribution

Suppose $a > 0$, $b > 0$, and

- $U \sim \chi_a^2$,
- $V \sim \chi_b^2$,
- U and V are independent.

Then

$$X = \frac{U/a}{V/b}$$

has an F distribution with a and b degrees of freedom.

7 Multivariate Normal Distributions

7.1 Goal

Generate $X \sim N_p(\mu, \Sigma)$, where $N_p(\mu, \Sigma)$ denotes the p -dimensional normal distribution with

- mean vector $\mu = (\mu_1, \dots, \mu_p)'$,
- variance-covariance matrix (symmetric and non-negative definite)

$$\Sigma = \begin{pmatrix} \sigma_1^2 & \sigma_{12} & \dots & \sigma_{1p} \\ \sigma_{12} & \sigma_2^2 & \dots & \sigma_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_{1p} & \sigma_{2p} & \dots & \sigma_p^2 \end{pmatrix}.$$

Here we use X, Y, Z to denote vectors, and $\mathbf{X}, \mathbf{Y}, \mathbf{Z}$ to denote matrices.

7.2 Review of random matrices

- Let $\mathbf{Y}$ denote an $n \times p$ random matrix. Let $\mathbf{B} \in \mathbb{R}^{m \times n}$, $\mathbf{C} \in \mathbb{R}^{p \times q}$, and $\mathbf{D} \in \mathbb{R}^{m \times q}$ be non-random. Then

$$\mathbf{E}(\mathbf{B}\mathbf{Y}\mathbf{C} + \mathbf{D}) = \mathbf{B} \mathbf{E}(\mathbf{Y}) \mathbf{C} + \mathbf{D}.$$

- Let X denote a p -dimensional random vector and define $Y = \mathbf{B}X + b$, where $\mathbf{B} \in \mathbb{R}^{q \times p}$ and $b \in \mathbb{R}^q$ are non-random. Then

$$\mathbf{E}(Y) = \mathbf{B} \mathbf{E}(X) + b, \quad \text{Var}(Y) = \mathbf{B} \text{Var}(X) \mathbf{B}'.$$

- Suppose that $W \sim N_p(\mu, \Sigma)$, $\mathbf{B} \in \mathbb{R}^{k \times p}$ is non-random, and $b \in \mathbb{R}^k$ is non-random. Then

$$Y = \mathbf{B}W + b \sim N_k(\mathbf{B}\mu + b, \mathbf{B}\Sigma\mathbf{B}').$$

7.3 Eigen-decomposition

- For a $p \times p$ symmetric matrix $\mathbf{A}$ we can write $\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}'$, where $\mathbf{V} \in \mathbb{R}^{p \times p}$ is an orthogonal matrix (hence $\mathbf{V}'\mathbf{V} = \mathbf{I}_p$) and $\mathbf{\Lambda} \in \mathbb{R}^{p \times p}$ is diagonal.
- The columns of $\mathbf{V}$ are called the **eigenvectors** of $\mathbf{A}$ and the diagonal elements of $\mathbf{\Lambda}$ are called the **eigenvalues** of $\mathbf{A}$, ordered $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p$.
- The matrix $\mathbf{A}$ is **positive semi-definite** if and only if $\lambda_p \geq 0$. It is **positive definite** if and only if $\lambda_p > 0$.

The eigen-decomposition allows for much easier computation of power series of matrices. Suppose $\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}'$ with $\lambda_1, \dots, \lambda_p \geq 0$. We have

$$\mathbf{A}^x = \mathbf{V}\text{diag}(\lambda_1^x, \dots, \lambda_p^x)\mathbf{V}' = \mathbf{V}\mathbf{\Lambda}^x\mathbf{V}',$$

where x is any real number. For example,

$$\mathbf{A}^{1/2} = \mathbf{V}\text{diag}(\lambda_1^{1/2}, \dots, \lambda_p^{1/2})\mathbf{V}' = \mathbf{V}\mathbf{\Lambda}^{1/2}\mathbf{V}',$$

because

$$\begin{aligned}\mathbf{A}^{1/2}\mathbf{A}^{1/2} &= \mathbf{V}\mathbf{\Lambda}^{1/2}\mathbf{V}'\mathbf{V}\mathbf{\Lambda}^{1/2}\mathbf{V}' \\ &= \mathbf{V}\mathbf{\Lambda}^{1/2}\mathbf{\Lambda}^{1/2}\mathbf{V}' \\ &= \mathbf{V}\mathbf{\Lambda}\mathbf{V}' = \mathbf{A}.\end{aligned}$$

If matrix $\mathbf{A}$ can be eigen-decomposed and none of its eigenvalues are zero, then $\mathbf{A}$ is non-singular and its inverse is $\mathbf{A}^{-1} = \mathbf{V}\mathbf{\Lambda}^{-1}\mathbf{V}'$.

7.4 Generate one copy from $N_p(\mu, \Sigma)$

Goal: generate $X \sim N_p(\mu, \Sigma)$.

- **Algorithm:**
 1. Generate a p -dimensional vector $Z = (z_1, \dots, z_p)$ where each $z_j \sim N(0, 1)$ independently (say, using the polar method).
 2. Let $X = \mu + \Sigma^{1/2}Z$.
- This is because $E(X) = \mu + \Sigma^{1/2}E(Z) = \mu$ and $\text{Var}(X) = \Sigma^{1/2}\text{Var}(Z)\Sigma^{1/2} = \Sigma^{1/2}\mathbf{I}_p\Sigma^{1/2} = \Sigma$.

```
set.seed(5400)
# sample size and dimension
n <- 1000
p <- 5
# var-covariance matrix
Sigma <- matrix(0.5, p, p)
diag(Sigma) <- 1
mu <- seq(p) # mean vector
```

```
# eigen-decomposition
eig <- eigen(Sigma)
Sigma.sqrt <- eig$vectors %*%
  tcrossprod(diag(sqrt(eig$values)), eig$vectors)
```

```
Z <- rnorm(p)
X <- mu + Sigma.sqrt %*% Z
X
```

7.5 Generate multiple copies from $N_p(\mu, \Sigma)$

Goal: generate $X_1, \dots, X_n \stackrel{\text{i.i.d.}}{\sim} N_p(\mu, \Sigma)$.

- **Algorithm:**

1. Generate an $n \times p$ random matrix $\mathbf{Z}$ whose entries are i.i.d. $N(0, 1)$.
2. Let $\mathbf{X} = \mathbf{1}_n \mu' + \mathbf{Z} \Sigma^{1/2}$. Then $\mathbf{X}$ is an $n \times p$ matrix whose rows are i.i.d. $N_p(\mu, \Sigma)$.

- You may either generate those random variables one by one, or use a more efficient “vectorized” way. Think about how $\mathbf{Z} \Sigma^{1/2}$ works.

7.5.1 R

```
set.seed(5400)
# each row of Z is from N(0, 1)
Z <- matrix(rnorm(n * p), n, p)
# each row of X is from N(mu, Sigma)
X <- tcrossprod(rep(1, n), mu) + Z %*% Sigma.sqrt
```

```
(Xbar <- colMeans(X))
```

```
(S <- cov(X))
```

7.5.2 Python

```
import numpy as np

np.random.seed(5400)
n = 1000
p = 5
mean = [x + 1 for x in range(p)]
sigma = [[0.5 for i in range(p)] for j in range(p)]
for i in range(p):
    sigma[i][i] = 1

v, w = np.linalg.eigh(sigma)
sigma_sqrt = w @ np.diag([v[i]**0.5 for i in range(p)]) @ w.T
Z = np.random.normal(0, 1, n * p).reshape(n, p)
mean_v = np.tile(mean, (n, 1))
X = mean_v + Z @ sigma_sqrt

print(np.mean(X, 0))
print(np.cov(X.T))
```

Note the order: $\mathbf{Z}\Sigma^{1/2}$ with the matrix on the *right*, because each row of $\mathbf{Z}$ is one observation. Writing $\Sigma^{1/2}\mathbf{Z}$ gives a dimension error, because $\mathbf{Z}$ is $n \times p$.

7.6 R library mvtnorm

```
set.seed(5400)
Z <- rnorm(p)
X <- mu + Sigma.sqrt %*% Z
X[1:2]
```

```
set.seed(5400)
library(mvtnorm)
X <- rmvnorm(n = 1, mean = mu, sigma = Sigma)
X[1:2]
```

7.7 Python function `multivariate_normal`

```
np.random.seed(5400)
Z = np.random.normal(0, 1, p)
X = mean + Z @ sigma_sqrt
print(X[0:2])
```

```
from scipy.stats import multivariate_normal

np.random.seed(5400)
d = multivariate_normal(mean=mean, cov=sigma)
X = d.rvs(1)
print(X[0:2])
```

The two Python results disagree, because $\Sigma^{1/2}$ is not unique. The symmetric root and the Cholesky factor both give the right distribution, but different numbers.